

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ



Báo cáo kết thúc học phần

Running Symbolic Execution Forever

Tên môn học:	Các vấn đề hiện đại của Công nghệ thông tin
Mã học phần:	INT3507
Sinh viên thực hiện:	Trịnh Văn Khánh - 21020343 Lê Phương Linh - 21020544 Trần Quý Mạnh - 21020352 Lê Duy Quang - 21020380
Giảng viên:	TS.Nguyễn Đức Anh

Hà Nội, 2024

LỜI CAM ĐOAN

Nhóm chúng em xin cam kết quá trình thực hiện nội dung báo cáo không sử dụng các công cụ trí tuệ nhân tạo tạo sinh như ChatGPT hay những công cụ tạo sinh tự động khác, cũng như không sao chép từ các sản phẩm khác. Nếu vi phạm, chúng em xin hoàn toàn chịu trách nhiệm và nhận 0 điểm cho học phần này.

Hà Nội, ngày 19 tháng 12 năm 2024

Các thành viên nhóm

Phân chia công việc

Họ và tên	Mã sinh viên	Phần trăm	Công việc
Trịnh Văn Khánh	21020343	26%	Slide trình chiếu, trình bày, trả lời phản biện
Lê Phương Linh	21020544	25,5%	Tìm hiểu các nội dung chính, trình bày nội dung báo cáo
Trần Quý Mạnh	21020352	24%	Thực hiện thực nghiệm trên công cụ của các tác giả, đóng góp nội dung phần thực nghiệm
Lê Duy Quang (Nhóm trưởng)	21020380	24,5%	Cài đặt kĩ thuật phục vụ cho minh họa, đóng góp nội dung phần minh họa, trả lời phản biện

Mục lục

Lời cam đoan	i
Danh sách hình vẽ	v
Danh sách bảng	vi
Danh sách thuật toán	vii
Danh sách mã nguồn	viii
Các từ viết tắt	ix
Mở đầu	x
1 Giới thiệu	1
1.1. Bối cảnh	1
1.2. Bài toán	2
1.3. Ứng dụng	3
2 Kiến thức nền tảng	4
2.1. Thực thi tượng trưng	4
2.2. Memoization	5
3 Phương pháp Running Symbolic Execution Forever	7
3.1. Mô tả chi tiết	7
3.1.1. Ghi nhớ kết quả bộ giải	8
3.1.2. Ghi nhớ và thực hiện lại	9
3.1.3. Phát hiện phân kỳ	11

3.2.	Các vấn đề đã giải quyết	14
3.2.1.	Thay đổi heuristics tìm kiếm	14
3.2.2.	Tầm nhìn toàn cục trong khi chạy	15
3.2.3.	Tương tác trực tiếp với môi trường	15
3.2.4.	Cắt giảm đường đi	16
4	Thực nghiệm và kết quả	17
4.1.	Câu hỏi thực nghiệm	17
4.2.	Thiết lập thực nghiệm	17
4.2.1.	Cấu hình	17
4.2.2.	Dataset	18
4.3.	Tăng tốc độ tái thực thi	18
4.3.1.	Ghi lại Coreutils bằng RndCov	19
4.4.	Symbolic execution chạy trong thời gian dài	21
5	Minh hoạ	24
6	Kết luận	29

Danh sách hình vẽ

2.1	Chương trình và cây thực thi	5
3.1	Phân loại nút trên cây thực thi	8
3.2	Thực thi lại các đường gián đoạn	16
4.1	Phân phối thời gian chạy của 37 ứng dụng coreutils	20
4.2	Tổng thời gian chạy của 37 ứng dụng coreutils	21
4.3	Số lượng lệnh được chạy lại trong mỗi lần tái thực thi	22
4.4	Số lượng lệnh mới (chưa được ghi nhớ) trong mỗi lần tái thực thi	23
4.5	Số lượng lệnh được chạy lại trong mỗi lần tái thực thi	23
5.1	Cây thực thi ghi nhớ sau lần chạy ban đầu	26
5.2	Cây thực thi ghi nhớ sau lần chạy thứ hai	28

Danh sách bảng

5.1	Các đường thi hành được thực thi trong lần chạy ban đầu	26
5.2	Các đường thi hành được thực thi trong lần chạy thứ hai	27

Danh sách thuật toán

1	Hàm kiểm tra tính thoả mãn <i>isSAT</i>	9
2	Thuật toán kiểm tra điều kiện nhánh	10
3	Thuật toán phát hiện thay đổi	13

Danh sách mã nguồn

5.1	Ví dụ một hàm TypeScript minh họa cho kĩ thuật	25
5.2	Hàm TypeScript sau khi đã thay đổi	27

Các từ viết tắt

Từ viết tắt	Tên đầy đủ
LOC	Line Of Code
PC	Path Constraints
SAT	SATisfiability
DFS	Depth First Search
BFS	Breadth First Search
ID	IDentification
CPU	Central Processing Unit
RAM	Random Access Memory

Mở đầu

Trong hoạt động kiểm thử phần mềm, các ca kiểm thử thường được tạo ra một cách thủ công, gây tốn kém về chi phí cũng như thời gian để hoàn thành công đoạn này. Thực thi tượng trưng (symbolic execution)[9] được biết đến là một kỹ thuật nổi tiếng với khả năng tự động sinh những ca kiểm thử có độ bao phủ cao với các tiêu chí kiểm thử nhằm phát hiện những lỗi sâu trong hệ thống phần mềm phức tạp. Triển khai và áp dụng sinh ca kiểm thử tự động giúp quá trình kiểm thử trở nên dễ dàng và nâng cao chất lượng sản phẩm.

Chủ đề “*Running Symbolic Execution Forever*” [3] trình bày những nhược điểm còn tồn tại của phương pháp thực thi giá trị tượng trưng. Trong đó, trở ngại về bộ nhớ sử dụng là vấn đề tiên quyết ảnh hưởng đến thời gian và tài nguyên khi kiểm thử.

Bài báo giới thiệu một kỹ thuật có thể ghi lại quá trình phân tích thực thi đang diễn ra vào ổ đĩa và khôi phục có chọn lọc đường thực thi sau đó, giúp quá trình này có thể chạy liên tục vô thời hạn. Đồng thời, phát hiện sự sai lệch khi thực thi lại và thay đổi chiến lược tìm kiếm tối ưu theo trường hợp khác nhau. Kỹ thuật trên được thực nghiệm hiệu quả trên 93 ứng dụng Linux, cho phép các ứng dụng này chạy trong nhiều ngày trong khi vẫn tiếp tục khám phá đường dẫn thực thi mới.

Nội dung báo cáo được chia thành 6 chương, bao gồm:

- *Chương 1:* Giới thiệu tổng quan về bối cảnh bài toán và các nghiên cứu trước đây.
- *Chương 2:* Trình bày kiến thức cơ sở được áp dụng trong phương pháp.
- *Chương 3:* Trình bày các kỹ thuật được triển khai trong chủ đề “*Running Symbolic Execution Forever*”
- *Chương 4:* Mô tả quá trình thực nghiệm của nhóm thực hành, so sánh kết quả với nhóm tác giả.

- *Chương 5*: Minh hoạ phương pháp bằng việc cài đặt vào một công cụ sinh dữ liệu kiểm thử tự động cho ngôn ngữ TypeScript.
- *Chương 6*: Tổng quan nội dung bài báo cáo, đưa ra kết luận và phụ lục.

Chương 1

Giới thiệu

Trong chương 1, báo cáo giới thiệu tổng quan về phương pháp thực thi chương trình bằng giá trị tương trưng, đồng thời đưa ra một số ứng dụng thực tiễn.

1.1 Bối cảnh

Hiện nay có nhiều phương pháp khác nhau hỗ trợ việc sinh tự động các ca kiểm thử giúp giảm chi phí và thời gian thực hiện đã được nghiên cứu và đưa ra ví dụ như Sound Static Analysis [7], thực thi tượng trưng. Cùng với việc thực thi tượng trưng đang ngày càng phổ biến, các nghiên cứu chỉ ra rằng đi kèm với khả năng sinh những bộ test case hiệu quả với có độ bao phủ cao và lỗi được phát hiện đúng của thực thi tượng trưng là hạn chế về tốn kém tài nguyên. Khi áp dụng thực thi tượng trưng trong phân tích ứng dụng trong thế giới thực, việc liệt kê và kiểm tra kỹ lưỡng mọi đường dẫn tiềm năng thường sử dụng hết bộ nhớ khả dụng trong một khoảng thời gian tương đối ngắn. Điều này dẫn đến gián đoạn và ứng dụng không thể phân tích trong một thời gian dài.

Nhiều phương pháp khác nhau đã được nghiên cứu và đưa ra nhằm hỗ trợ giải quyết vấn đề bùng nổ số lượng đường thực thi. Một trong số đó là lưu trữ đường đi bị kết thúc sớm vào đĩa và chạy lại chúng trong lần tái thực thi. Cụ thể, phương pháp Memoise[12] ghi các đường thực thi vào đĩa dưới dạng trie và chỉ đưa trở lại bộ nhớ đường đi cần quan tâm. Cùng với đường đi, kết quả giải điều kiện ràng buộc được tái sử dụng để tăng tốc độ. Cách tiếp cận này đã được chứng minh

là có thể áp dụng để phân tích sâu hơn, phân tích hồi quy và cải thiện phạm vi bao phủ. Tuy nhiên cách làm trên có hạn chế là cần sử dụng cùng một heuristic tìm kiếm cho quá trình thực thi lại và chỉ được áp dụng cho ứng dụng Java có phạm vi nhỏ (<5000 LOC) với các lần chạy ngắn.

1.2 Bài toán

Để khắc phục những hạn chế của phương pháp memoization, *Running Symbolic Execution Forever* ghi lại thông tin khôi phục đường thực thi tượng trưng đang diễn ra vào đĩa và gọi lại có chọn lọc đường thực thi quan tâm.

Điều này giúp chạy thực thi tượng trưng trên chương trình lớn vô thời hạn. Đồng thời tiếp tục khám phá những con đường mới bằng cách sử dụng với một phương pháp tìm kiếm heuristic bất kỳ trong mỗi lần tái thực thi. Phương pháp có mục tiêu giải quyết những thách thức nghiên cứu sau:

- **Khác biệt về môi trường trong giữa các lần thực thi:** Ứng dụng trong thế giới thực luôn tồn tại môi trường trong lần chạy ban đầu thay đổi so với lần chạy lại. Nếu không phát hiện những thay đổi, sử dụng lại đường đi đã ghi nhớ có thể dẫn đến khám phá đường đi không khả thi.
- **Lưu trữ số lượng lớn đường đi:** Phân tích một chương trình trong thời gian dài tương đương hàng triệu đường đi cần được lưu trữ vào đĩa. Cần phát triển một phương pháp lưu trữ đường đi hiệu quả, đồng thời lưu giữ đủ thông tin để phát hiện sự khác biệt do môi trường gây ra.
- **Cung cấp góc nhìn tổng quát của cây thực thi chương trình** trong khi khôi phục toàn bộ hàng triệu nút bộ nhớ là không khả thi.
- **Khắc phục hạn chế sử dụng cùng một phương pháp tìm kiếm** đối với lần thực thi ban đầu và thực thi lại. Đây là điều cần thiết để tái tạo kết quả do cây gốc tạo ra, thêm vào đó, có thể tăng tốc độ tái thực thi và cho phép thay đổi động các phương pháp tìm kiếm khi cần thiết.

1.3 Ứng dụng

- **Phân tích chương trình:** Symbolic execution cho phép lập trình viên tự động tìm kiếm điều kiện hoặc mẫu hành vi trong hệ thống phần mềm, bao gồm việc phát hiện đoạn mã vi phạm ràng buộc hoặc đoạn mã không thể truy cập được. Ngoài ra, symbolic execution có khả năng suy diễn ràng buộc mới từ cây thực thi của chương trình, giúp lập trình viên quét tình huống đặc biệt hoặc lỗ hổng bảo mật. Công cụ symbolic execution có thể tìm đường dẫn dẫn đến sự cố chương trình hoặc phát hiện hành vi đáng ngờ như truy cập vào vùng nhớ chưa được khởi tạo. Một số công cụ sử dụng symbolic execution khác có thể tạo ra trường hợp khai thác lỗ hổng, giúp tái hiện lỗi đã phát hiện [2], [4], [6].
- **Kiểm thử phần mềm:** Symbolic execution hỗ trợ tạo tập dữ liệu đầu vào cho tất cả đường dẫn thực thi được bao phủ trong chương trình. Các công cụ kiểm thử symbolic cho phép khai báo điều kiện đầu vào (preconditions) và điều kiện sau (postconditions), sau đó kiểm tra tự động để tìm ra đầu vào vi phạm điều kiện đã định. Trong đó, framework PEX trong C# là ví dụ cho phép viết kiểm thử tham số hóa, đảm bảo đầu ra của chương trình luôn hợp lệ đối với mọi đầu vào [1].
- **Phân tích và tái hiện:** Symbolic execution được ứng dụng trong *Reverse Engineering* bằng cách cung cấp biểu diễn thay thế để giúp lập trình viên hiểu hành vi của mã nguồn. Tiêu biểu có công cụ Microsoft IntelliTest [8], tạo ra bảng tóm tắt gồm tập hợp đầu vào, giá trị trả về, và các biểu thức liên quan từ cây thực thi symbolic. Điều này giúp lập trình viên dễ dàng khám phá trường hợp đặc biệt mà không cần đọc toàn bộ mã nguồn. Ngoài ra, công cụ debug symbolic cho phép lập trình viên duyệt cây thực thi, khám phá trạng thái symbolic, và kiểm tra mã theo từng bước.

Chương 2

Kiến thức nền tảng

Trong chương 2, báo cáo làm rõ các khái niệm về thực thi tượng trưng và trình bày chi tiết về các kỹ thuật được sử dụng trong giải quyết bài toán.

2.1 Thực thi tượng trưng

Thực thi tượng trưng là khám phá tất cả đường thực thi của chương trình không sử dụng các giá trị cụ thể (concrete value). Tất cả tính toán thực hiện trên giá trị tượng trưng của tham số đầu vào. Giá trị đầu ra của chương trình biểu diễn bởi một biểu thức tượng trưng.

Chương trình khởi tạo đường đi, thực hiện tính toán và cập nhật trên giá trị tượng trưng. Đường thực thi rẽ hai nhánh *True* và *False* khi gặp câu lệnh điều kiện, đồng thời bổ sung vào ràng buộc đường đi ràng buộc tương ứng với hai nhánh được tạo ra. Một ràng buộc là một biểu thức Boolean (không định lượng). Tập hợp các ràng buộc là điều kiện tương ứng với đường đi. Giải ràng buộc này sẽ sinh ra giá trị cụ thể cho các tham số đầu vào tương ứng với từng nhánh thực thi của chương trình. Bộ giải sẽ đánh giá xem đường đi tiếp theo có khả thi hay không, tương đương với kiểm tra tồn tại bộ giá trị thỏa mãn. Nếu không thỏa mãn thì ràng buộc sẽ đánh giá là sai, khi đó thực thi tượng trưng sẽ dừng hoặc quay lui thực thi theo nhánh khả thi.

Trên thực tế, đối với những đoạn mã chứa vòng lặp hoặc lời gọi đệ quy cần phải thêm tiêu chí giới hạn cho việc tìm kiếm như thời gian thực thi, giới hạn chiều

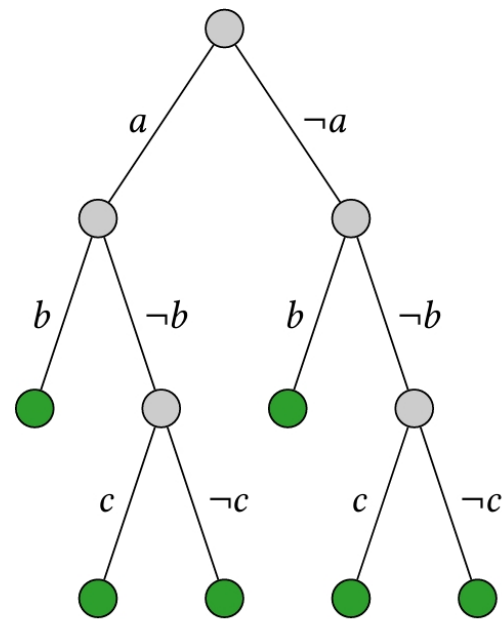
sâu của đường thực thi hoặc số lượng vòng lặp lồng nhau.

Tập hợp tất cả các đường thực thi tạo thành một cấu trúc cây gọi là cây thực thi (execution tree). Nút biểu diễn trạng thái và cung biểu diễn chuyển đổi giữa các trạng thái.

Trạng thái (state) bao gồm giá trị (tượng trưng) của biến chương trình và một ràng buộc đường đi (PC) của các đường đang chờ xử lý trong bộ nhớ.

Chỉ dẫn (instruction) là khối cơ bản chi phối việc thực hiện chương trình trên bộ xử lý.

```
1 int main(void) {  
2   int a, b, c; // symbolic  
3  
4   if (a) { ... }  
5   else { ... }  
6  
7   if (b) { ... }  
8   else {  
9     if (c) { ... }  
10    else { ... }  
11  }  
12 }
```



Hình 2.1: Chương trình và cây thực thi

2.2 Memoization

Memoization là một kỹ thuật lưu trữ và tái sử dụng kết quả của các tính toán tốn kém trong khi thực hiện thực thi tượng trưng. Thông tin lưu trữ bao gồm chuỗi các quyết định được thực hiện trong quá trình khám phá đường đi.

Phương pháp memoization [12] được triển khai dựa trên hai tiêu chí:

- Một là, nếu một ràng buộc đường dẫn (path constraint) đã được giải quyết trong lần thực thi trước, kết quả giải quyết có thể được tái sử dụng mà không

cần giải lại.

- Hai là, nếu ràng buộc đường dẫn cho tất cả các nhánh tiếp theo trong không gian tìm kiếm vẫn giống như lần trước, không gian con bắt đầu từ điểm đó có thể bị cắt bỏ, và kết quả từ lần thực thi trước có thể được tái sử dụng.

Chương 3

Phương pháp Running Symbolic Execution Forever

Trong chương 3, báo cáo trình bày chi tiết về đề tài và hiệu quả giải quyết của phương pháp đề xuất.

3.1 Mô tả chi tiết

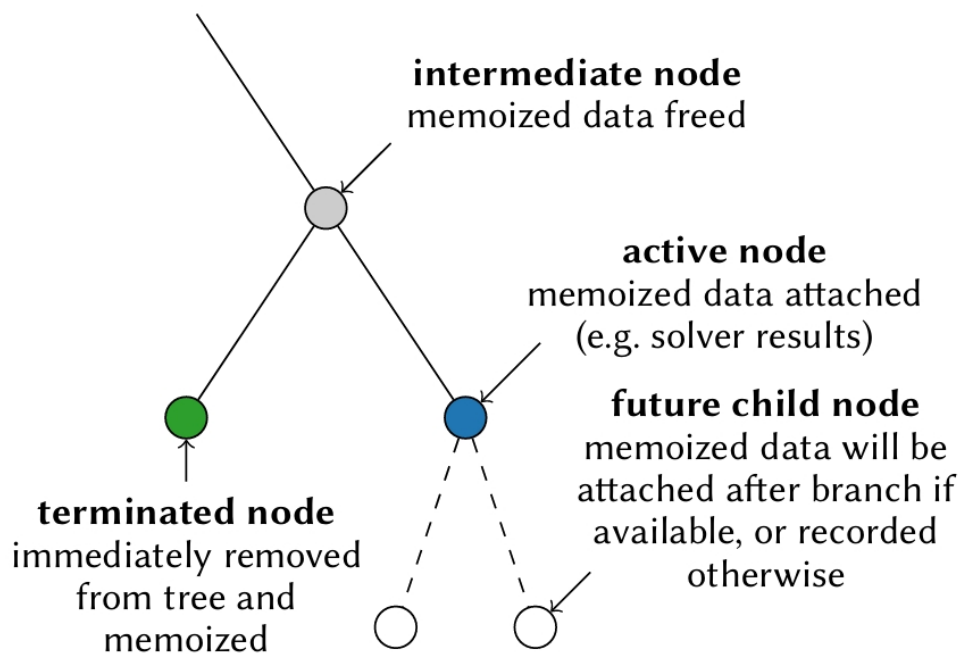
Trong thực thi tượng trưng "cổ điển", kết quả thực thi tượng trưng cho một hệ thống không thể được sử dụng lại trong phân tích tiếp theo trên hệ thống đó. Người dùng phải chạy lại thực thi tượng trưng từ đầu ngay cả khi hệ thống không có bất kỳ thay đổi nào. Bài báo trình bày một framework làm giảm đáng kể thời gian tái thực thi bằng cách ghi nhớ kết quả của bộ giải và cắt bớt cây con được khám phá đầy đủ khỏi lần thực thi lại. Phát hiện phân kỳ đảm bảo rằng đường dẫn được thực hiện lại thực hiện chỉ dẫn giống với đường thực thi đã được ghi lại của chúng.

Trong khi tái thực thi, trước tiên sẽ phân tích cây thực thi để đánh dấu đường dẫn cần thực thi lại - tất cả nút trên đường dẫn từ gốc đến các nút "ứng viên" được đánh dấu là "đã bật". Khi thực hiện lặp lại để khám phá cây thực thi, bất cứ khi nào một lệnh điều kiện được thực thi tượng trưng, nó sẽ thực hiện phép duyệt tương ứng trong cây thực thi. Phương pháp bỏ qua giải ràng buộc cho phần đường đi có thông tin đã được lưu trữ trong trie.

3.1.1 Ghi nhớ kết quả bộ giải

Nhằm giải quyết vấn đề về thời gian tái thực thi, phương pháp ứng dụng memoise. Trên mỗi nút, lưu một ID, ID này sử dụng để tham chiếu. Nút cha biết được ID có khả năng được gán cho nút con trực tiếp. Nút trên cây được phân loại làm nút hoạt động, nút trung gian, nút cần xóa và nút con tương lai.

Tất cả nút hoạt động được chú thích bằng siêu dữ liệu cần thiết để thực thi đường đi từ nhánh cuối cùng của nó. Siêu dữ liệu bao gồm kết quả của bộ giải, số lượng instruction đã thực hiện, nhánh tượng trưng và khối băm cơ bản, ngoài ra còn có thông tin gỡ lỗi và thống kê cơ bản.



Hình 3.1: Phân loại nút trên cây thực thi

Nút trung gian là tất cả nút nằm giữa nút gốc và nút hoạt động biểu diễn đường thực thi để đạt đến trạng thái của nút hoạt động. Nút trung gian chỉ biểu diễn duy nhất đường đi có thể xảy ra. Khi trạng thái lưu trên nút hoạt động kết thúc hoặc tới nhánh tượng trưng, nút hoạt động trở thành nút trung gian còn nút con tương lai trở thành nút hoạt động tiếp theo. Chương trình thực hiện giải phóng dữ liệu ở nút cha và nút cha trở thành nút trung gian.

3.1.2 Ghi nhớ và thực hiện lại

Sử dụng dữ liệu đã lưu cho thực thi lại tương đương liên kết dữ liệu đã ghi nhớ với lần chạy mới. Cần kiểm tra tính đúng đắn của trạng thái đang thực thi để việc liên kết dữ liệu không dẫn đến sai lệch. Chương trình kiểm tra tính đúng đắn của trạng thái đang thực thi bằng cách giải điều kiện. Nếu trạng thái thỏa mãn điều kiện, đơn giản gọi lại và gán dữ liệu đã lưu cho nút.

Thuật toán 1 Hàm kiểm tra tính thỏa mãn *isSAT*

```
1: global Solver
2: Function isSAT(state, condition)
3: if inReexecutionMode(state) then
4:   if state.node.solverResults.hasNext() then
5:     return state.node.solverResults.getNext()
6:   end if
7: end if
8: result  $\leftarrow$  Solver.isSAT(state.constraints, condition)
9: state.node.solverResults.append(result)
10: return result
```

Hàm *isSAT* trên nhận trạng thái và điều kiện. Trạng thái bao gồm cả tham chiếu *state.node* đến nút tương ứng trong cây thực thi, trả về kết quả kiểm tra thỏa mãn điều kiện. Sau khi kiểm tra kết quả, nếu đoạn mã thỏa mãn điều kiện đã từng thực thi trước và kết quả đã lưu vào bộ nhớ đệm, thuật toán sẽ lấy kết quả đã lưu sẵn từ nút tương ứng với trạng thái hiện tại (dòng 3 - 5). Nếu không có kết quả được lưu, thuật toán sẽ gọi bộ giải cơ bản (underlying solver) ở dòng 8 để tính toán kết quả cho điều kiện trong trạng thái hiện tại. Sau khi bộ giải có kết quả, kết quả này sẽ được lưu lại trong nút tương ứng với trạng thái hiện tại (dòng 9). Kết quả đã lưu tiếp tục được sử dụng để kiểm tra lại điều kiện trong trạng thái này ở lần sau mà không cần tính toán lại.

Khi một trạng thái đến một nhánh tượng trưng (symbolic branch), cập nhật cây thực thi.

Hàm nhận được trạng thái và điều kiện, trả về output là hai trạng thái con:

- **trueState:** Trạng thái trong trường hợp điều kiện nhánh đúng. Nếu nhánh này không khả thi, trueState sẽ là NULL.
- **falseState:** Trạng thái khi điều kiện rẽ nhánh là phủ định. Nếu nhánh này

Thuật toán 2 Thuật toán kiểm tra điều kiện nhánh

```
1: Function branch(state, condition)
2: trueSAT  $\leftarrow$  isSAT(state, condition)
3: falseSAT  $\leftarrow$  isSAT(state,  $\neg$ condition)
4: bothFeasible  $\leftarrow$  trueSAT  $\wedge$  falseSAT
5: if  $\neg$ bothFeasible then
6:   comment: Only one side is feasible, continue with current state
7:   if trueSAT then
8:     checkOrAppendBranch(state, true)
9:     Return (state, NULL)
10:  else
11:    checkOrAppendBranch(state, false)
12:    Return (NULL, state)
13:  end if
14: else
15:   comment: Both branches feasible
16:   checkOrSetFork(state)
17:   trueState  $\leftarrow$  createNewState(state, condition)
18:   if state.node.trueID then
19:     trueState.node  $\leftarrow$  initFromDB(state.node.trueID)
20:   else
21:     trueState.node  $\leftarrow$  createNewActiveNode()
22:   end if
23:   state.node.trueID  $\leftarrow$  trueState.node.ID
24:   ... comment: Similarly for falseState
25:   writeNodeToDB(state.node)
26:   freeData(state.node)
27:   Return (trueState, falseState)
28: end if
```

không khả thi, *falseState* sẽ là *NULL*.

Thuật toán tiếp tục kiểm tra tính khả thi của điều kiện nhánh (*dòng 2*) và phủ định của nó (*dòng 3*). Nếu chỉ có một nhánh khả thi, nghĩa là không cần phân nhánh, trạng thái hiện tại và nút hiện tại vẫn tiếp tục được sử dụng mà không cần cập nhật trên cơ sở dữ liệu (*dòng 5 đến 13*). Nếu cả hai nhánh đều khả thi, cập nhật cây thực thi với trạng thái mới được tạo (*dòng 14-16*). Thuật toán kiểm tra xem nút hiện tại đã tồn tại *trueID*, nghĩa là đã có trạng thái con tương ứng khi điều kiện đúng trong cơ sở dữ liệu (*dòng 17*). Nếu có, liên kết trạng thái mới với nút tương ứng từ cơ sở dữ liệu (*dòng 18*). Nếu không, tạo một nút con mới với ID

duy nhất (dòng 20). trueID của nút hiện tại trở đến nút mới (dòng 22). Lặp lại các bước trên với trạng thái còn lại (dòng 23). Sau khi cập nhật *state.node* vào cơ sở dữ liệu (dòng 24), thực hiện giải phóng khỏi bộ nhớ (dòng 25) và trả về trạng thái con (dòng 26).

3.1.3 Phát hiện phân kỳ

Trong thực tế, các công cụ không sử dụng thực thi tượng trưng cho toàn bộ quá trình kiểm thử mà sử dụng kết hợp thực thi cụ thể và thực thi tượng trưng. Nghĩa là chương trình đang được kiểm thử vẫn có thể tương tác trực tiếp với môi trường, chẳng hạn như gọi thư viện hoặc thực hiện system calls của hệ điều hành. Trong quá trình tái thực thi, những thay đổi về môi trường như dấu thời gian của tệp hoặc dung lượng bộ nhớ khả dụng có thể làm cho đường đi chệch khỏi đường thực thi ban đầu. Thực thi tượng trưng thực hiện xen kẽ việc thực thi đường đi khác nhau, thay đổi thứ tự thực thi bằng cách sử dụng một phương pháp tìm kiếm khác vì vậy có thể dẫn tới một khám phá khác. Để đảm bảo hiệu suất, việc thực thi đường đi đơn lẻ không được cách ly hoàn toàn có thể dẫn đến mất mát thông tin của đường đi. Cuối cùng, chương trình thực thi tượng trưng thường thêm nguồn không xác định, ví dụ: sử dụng bảng băm được lập chỉ mục theo địa chỉ bộ nhớ cụ thể có thể khác nhau trong các lần thực thi.

Một thuộc tính quan trọng của thực thi tượng trưng là chỉ có đường đi khả thi được khám phá. Để bảo toàn thuộc tính này, đường đi được thực thi lại phải thực hiện cùng một chuỗi lệnh như đường đi đã ghi nhớ tương ứng của nó. Nếu không, bất kỳ sự phân kỳ nào cũng có thể thay đổi tập hợp điều kiện trên toàn bộ đường đi, dẫn đến làm mất hiệu lực kết quả giải được ghi nhớ, hoặc thậm chí sử dụng lại kết quả giải sai vị trí mã. Trong trường hợp đó, thực thi tượng trưng sẽ khám phá đường đi không khả thi, có khả năng dẫn đến kết quả sai. Để ngăn chặn những trường hợp như vậy, mỗi nút cây thực thi giữ thông tin tóm tắt quá trình thực thi giữa thời điểm nút được tạo ra lần đầu và thời điểm trạng thái của nó phân nhánh thành hai trạng thái con. Thông tin này gọi là thông tin bảo vệ bổ sung, bao gồm: số lượng lệnh, tổng băm của các khối cơ bản đã đi qua và vector hướng nhánh.

- Vector hướng nhánh (*true, false*): giá trị đầu tiên là hướng rẽ nhánh của

trạng thái hiện tại, các giá trị tiếp theo là *false* nếu duy nhất đường đi của nhánh *false* khả thi, hoặc *true* nếu duy nhất đường đi của nhánh *true* khả thi. Chuyển trạng thái mới khi đường đi có thể phân nhánh.

- Số lượng lệnh: đếm số lệnh đã thực thi trước khi trạng thái phân nhánh thành hai trạng thái.
- Tổng băm của tất cả khối cơ bản đã thực thi trước khi trạng thái phân nhánh giúp xác định đường đi đã được khám phá. Thông tin cập nhật dựa trên cập nhật băm của khối cơ bản của một trạng thái mỗi khi đường đi chạm tới một khối cơ bản mới và một map đã tính toán trước chứa tổng băm cho tất cả các khối.

Thuật toán 3 Thuật toán phát hiện thay đổi

```
1: Function updateSafeguardingData(state)
2: state.node.instructions  $\leftarrow$  state.instructionCounter
3: state.node.bbHash  $\leftarrow$  state.bbHash
4: state.node.branches  $\leftarrow$  state.branches
5: end function
6: Function checkOrAppendBranch(state, branch)
7: if inReexecutionMode(state) then
8:   if branch  $\neq$  state.node.branches.getNext() then
9:     markStateAsDiverging(state)
10:    updateSafeguardingData(state)
11:    state.node.branches.append(branch)
12:   end if
13: else
14:   state.node.branches.append(branch)
15: end if
16: end function
17: Function checkOrSetFork(state)
18: if inReexecutionMode(state) then
19:   i  $\leftarrow$  (state.node.instructions = state.instructions)
20:   h  $\leftarrow$  (state.node.bbHash = state.bbHash)
21:   b  $\leftarrow$   $\neg$ state.node.branches.hasNext()
22:   if  $\neg i \vee \neg h \vee \neg b$  then
23:     markStateAsDiverging(state)
24:     updateSafeguardingData(state)
25:   end if
26: else
27:   updateSafeguardingData(state)
28: end if
29: end function
```

Hàm *checkOrAppendBranch* so sánh nhánh được chọn trong quá trình tái thực thi giống với những nhánh được ghi nhớ trước đó trên đĩa. Hàm được gọi khi một nhánh tương trưng có một kết quả duy nhất (dòng 8, 11 thuật toán 2). Nếu đang tái thực thi trạng thái hiện tại, hàm sẽ kiểm tra việc thực thi nhánh (dòng 9). Nếu nhánh không được thực hiện như mong đợi, trạng thái sẽ được đánh dấu là phân nhánh (dòng 10), nghĩa là tất cả thông tin đã ghi nhớ sẽ bị loại bỏ và không được xét đến. Sau khi đánh dấu trạng thái là phân nhánh, hàm sẽ cập nhật thông tin bảo vệ (dòng 11) và thêm nhánh vào vector hướng nhánh (dòng 12) để theo dõi nhánh đã được khám phá trong quá trình thực thi. Trạng thái tiếp tục trong

recording mode, hàm chỉ cần thêm nhánh vào vector hướng nhánh không cần qua kiểm tra (dòng 15).

Khi một trạng thái sắp phân nhánh (dòng 14 thuật toán 2), hàm *checkOrSetFork* quyết định phân nhánh nếu nhận ra sự khác biệt giữa thông tin bảo vệ: số chỉ dẫn (dòng 21), tổng băm của khối cơ bản (dòng 22), vector hướng nhánh đã ghi lại cho đến thời điểm phân nhánh (dòng 23). Nếu không đúng (dòng 24), trạng thái sẽ được đánh dấu là phân nhánh (dòng 25) và cập nhật thông tin bảo vệ (dòng 26). Điều này có nghĩa là trạng thái đã rẽ ra khỏi những nhánh đã được ghi nhớ và do đó không còn khả năng dựa vào các kết quả đã lưu trữ. Trong recording mode, hàm chỉ đơn giản cập nhật thông tin bảo vệ liên quan đến nút trong cơ sở dữ liệu (dòng 29). Điều này có nghĩa là nhánh mới sẽ được ghi lại mà không cần thực hiện kiểm tra phức tạp.

Hạn chế: Thuật toán không phát hiện được tất cả phân kỳ. Các xung đột băm có thể bị bỏ sót. Đồng thời, thuật toán không thể phát hiện hành vi phân kỳ không làm thay đổi luồng điều khiển. Tuy nhiên, những tình huống như vậy khá hiếm và thuật toán khó có thể bỏ sót hoàn toàn sự phân kỳ vì mọi nhánh tương trưng đơn lẻ đều bị kiểm tra.

3.2 Các vấn đề đã giải quyết

3.2.1 Thay đổi heuristics tìm kiếm

Phương pháp Memoise lưu chuỗi đường đi đã được khám phá trong một cấu trúc dữ liệu cây có thứ tự dựa trên trie (không hẳn là trie). Nút con ứng với nhánh trong một câu lệnh điều kiện và được xếp thứ tự dựa trên thứ tự thực tế mà chúng được khám phá. Ví dụ nếu nhánh false được khám phá trước thì nút false sẽ xếp trước nút true. Trong lần chạy tiếp theo, cây được duyệt theo thứ tự và từ đây dẫn đến yêu cầu là chuỗi các đường đi phải giống với lần chạy trước, để cho thứ tự duyệt cây mới ứng với các đường đi này. Như vậy có nghĩa là bất kì chiến thuật khám phá (exploration strategy / heuristics) nào được sử dụng sẽ phải được giữ nguyên giữa tất cả lần chạy. Ở thuật toán 2, cơ sở dữ liệu ghi lại thông tin về cấu trúc của cây thực thi thông qua mối quan hệ giữa hai trạng thái con trueID và falseID, điều này tương đương thứ tự của các nhánh được khám phá khi thực hiện

heuristic tìm kiếm không nằm trong thông tin lưu trữ. Do đó kết quả được lưu trữ để tái thực thi hoàn toàn độc lập với heuristics tìm kiếm.

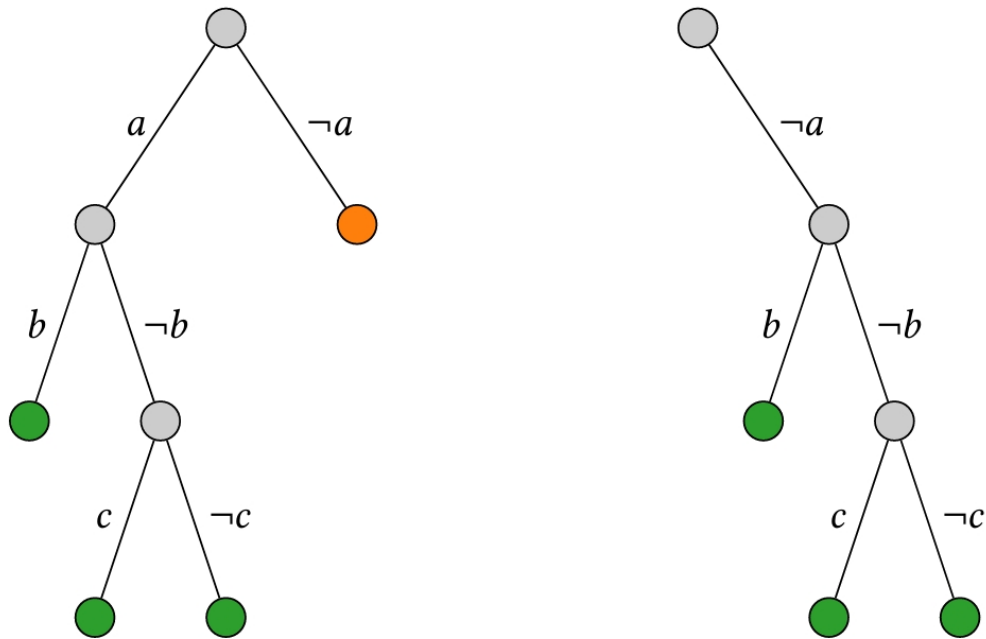
Khả năng sử dụng heuristic khác nhau đem đến nhiều lợi ích trong quá trình tối ưu hóa và cải thiện hiệu suất của hệ thống. Thứ nhất, nó tăng tốc độ tái thực thi. Sử dụng lại kết quả đã ghi nhớ giúp quá trình tái thực thi nhanh hơn. Với những phần đã ghi nhớ, có thể sử dụng heuristic ít tốn kém như tìm kiếm theo chiều sâu (DFS) và chỉ áp dụng heuristic tốn kém hơn cho những phần chưa được ghi nhớ. Thứ hai, nó giảm áp lực lên bộ nhớ. Trong khi tái thực thi, có thể thay heuristic tiêu tốn bộ nhớ như tìm kiếm theo chiều rộng (BFS) bằng phương pháp ít tốn bộ nhớ hơn như DFS, giúp quản lý bộ nhớ hiệu quả và tránh quá tải. Cuối cùng, có thể chỉ tái thực thi vào nhánh ưu tiên mà không cần phải tải toàn bộ cây lên bộ nhớ. Tài nguyên được sử dụng vào nhánh quan trọng, giúp tiết kiệm tài nguyên và tăng hiệu suất.

3.2.2 Tầm nhìn toàn cục trong khi chạy

Running symbolic execution forever lưu trữ một cây thực thi trên ổ đĩa qua siêu dữ liệu và liên tục bổ sung nhánh mới được khám phá. Siêu dữ liệu của đường đi chưa được khám phá đầy đủ trong quá trình tái thực thi lại được giữ lại trong khi đường đi mới được khám phá, tự động ghi lại siêu dữ liệu mới ngay khi chúng vượt quá dữ liệu đã ghi nhớ. Cây thực thi tiếp tục mở rộng cùng với việc chương trình được tái thực thi. Với mỗi lần tái thực thi, ta có một bức tranh hoàn chỉnh hơn về ứng dụng đang kiểm thử. Tầm nhìn toàn cục cho phép người ta xem xét một ứng dụng một cách kỹ lưỡng. Hơn nữa, biết được những đường dẫn nào đã được khám phá, việc thực hiện lại chúng để đánh giá thuộc tính khác nhau của ứng dụng trở nên dễ dàng hơn.

3.2.3 Tương tác trực tiếp với môi trường

So sánh với Memoise, phương pháp mới ghi nhớ và có thể phát hiện ra sự sai lệch giữa thông tin bảo vệ. Kết quả không bị ảnh hưởng do tính chất phụ thuộc môi trường chạy. Nhờ đó đảm bảo chỉ đường đi khả thi được khám phá trong khi tái thực thi.



Hình 3.2: Thực thi lại các đường gián đoạn

3.2.4 Cắt giảm đường đi

Ở phương pháp memoization, để sử dụng lại kết quả, cần kiểm tra ràng buộc đường đi cho tất cả các đường bắt đầu từ một điểm không thay đổi so với ràng buộc đường đi ở lần thực thi lại. Phương pháp mới thêm vào siêu dữ liệu lưu trữ trên nút cha trạng thái kết thúc của mỗi đường đi, ví dụ: thoát chương trình, tìm thấy lỗi, bị gián đoạn do áp lực bộ nhớ. Trong quá trình thực thi lại, khi một đường dẫn phân nhánh, chỉ cần kiểm tra duy nhất một giá trị này trên mỗi nhánh để xác định xem cây con của nó có chứa đường đi đã được khám phá đầy đủ hay không. Nếu đúng, chấm dứt việc khám phá đường đi trên nhánh tương ứng (siêu dữ liệu của nó vẫn được lưu trữ trong cơ sở dữ liệu). Bằng cách đó, chỉ thực thi lại đường dẫn bị gián đoạn.

Chương 4

Thực nghiệm và kết quả

Chương 4 trình bày kết quả thực nghiệm của tác giả và nhóm thực hiện, từ đó trả lời câu hỏi thực nghiệm trong bài báo.

4.1 Câu hỏi thực nghiệm

- **Câu hỏi 1:** Chương trình có giảm thời gian chạy của việc tái thực thi hay không?
- **Câu hỏi 2:** Chương trình có cho phép *symbolic execution* chạy trong thời gian dài và tiếp tục khám phá các đường chạy mới không?

4.2 Thiết lập thực nghiệm

4.2.1 Cấu hình

- **Tác giả:** Trong các thí nghiệm, nhóm tác giả đã sử dụng các máy tính đồng nhất với CPU Intel Core i7-4790 chạy ở tốc độ 3.6 GHz, RAM 16 GB và ổ cứng 1 TB (7200 vòng/phút). Tất cả các thí nghiệm được chạy trong các container Docker sử dụng cùng hệ điều hành với máy chủ Ubuntu 18.04.
- **Nhóm thực nghiệm:** Sử dụng máy tính với CPU Intel Core i5-10300 chạy ở tốc độ 2.5 GHz, RAM 16GB và ổ cứng 512 GB. Thí nghiệm chạy trong

container Docker với hệ điều hành Ubuntu 22.04.

4.2.2 Dataset

Nhóm tác giả sử dụng nhiều ứng dụng khác nhau: từ bộ sưu tập phần mềm GNU bao gồm các bộ Coreutils, diff, find và grep - là các ứng dụng hệ thống khá phức tạp, được hàng triệu người dùng sử dụng. Thêm vào đó, nhóm tác giả lựa chọn thêm các ứng dụng xử lý các định dạng đầu vào phức tạp hơn như hình ảnh, tệp nhị phân và gói dữ liệu mạng, tiêu biểu như: libspng, readelf và tcpdump.

GNU Coreutils Cung cấp nhiều công cụ khác nhau để thao tác với tệp, shell và văn bản. Bộ công cụ này được sử dụng trong bài báo giới thiệu KLEE. Bộ công cụ bao gồm 106 ứng dụng khác nhau, trong đó: Nhóm tác giả đã loại trừ năm công cụ tương tự base64 (base32, sha1sum, sha224sum, sha384sum, sha512sum) và một công cụ (cat) mà khi chạy với KLEE sẽ hết bộ nhớ chỉ sau vài giây do bộ đệm nội bộ lớn trong các phiên bản gần đây.

Ngoài ra, nhóm tác giả đã loại bỏ 13 công cụ (chcon, chgrp, chmod, chown, chroot, dd, ginstall, kill, rm, rmdir, stdbuf, truncate, unlink) do có hành vi không xác định khi thực hiện *symbolic execution*, vì việc thực thi các đường dẫn qua những công cụ này có thể ảnh hưởng đến việc thực thi các đường dẫn khác hoặc thậm chí ảnh hưởng đến hành vi của Docker và KLEE [5] mà không có các biện pháp cô lập bổ sung. Do đó, bộ công cụ còn lại tổng cộng 87 công cụ.

4.3 Tăng tốc độ tái thực thi

MoKlee triển khai hai cơ chế nhằm giảm chi phí thời gian chạy khi thực thi lại:

- Một là, *Ghi nhớ kết quả của bộ giải ràng buộc*.
- Hai là, *Cắt tỉa các đường dẫn*.

Để đánh giá hiệu quả của các cơ chế trên, nhóm tác giả đã chạy MoKlee ở chế độ record trên 93 ứng dụng để đo hiệu năng trong 2 giờ và đo thời gian cần thiết

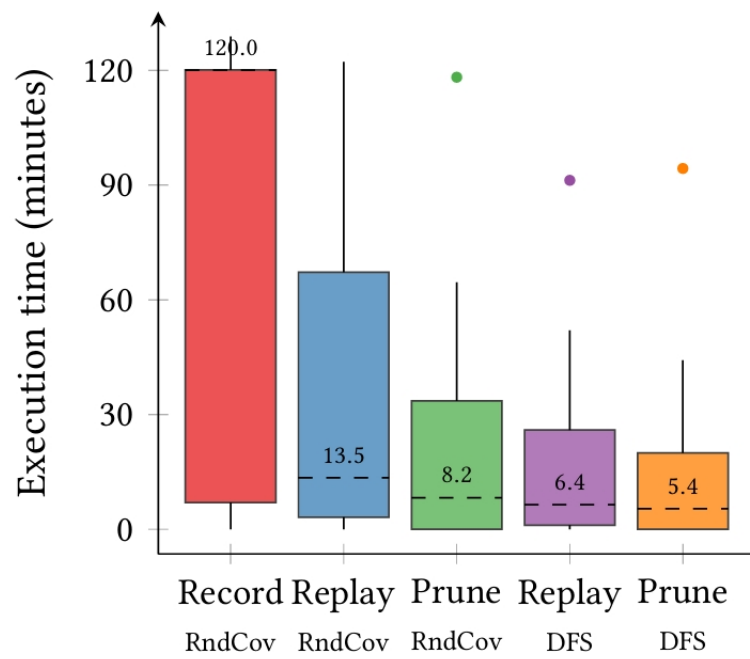
để thực thi lại các cây thực thi tương tự. Kết quả được đánh giá với hai phương pháp tìm kiếm heuristic khác nhau:

- **Tìm kiếm theo chiều sâu (DFS):** Tiết kiệm bộ nhớ.
- **Chiến lược mặc định của KLEE:** Kết hợp giữa lựa chọn đường dẫn ngẫu nhiên và tìm kiếm dựa trên phạm vi bao phủ (RndCov).

4.3.1 Ghi lại Coreutils bằng RndCov

Ban đầu, nhóm tác giả áp dụng cơ chế ghi nhớ (memoization) trên 87 công cụ Coreutils bằng cách sử dụng chiến lược khám phá mặc định (RndCov). Sau đó, thực thi lại các phiên này với cơ chế cắt tỉa đường dẫn (path pruning) bị vô hiệu hóa (replay) hoặc được kích hoạt (prune) bằng cách sử dụng mỗi chiến lược tìm kiếm (DFS, RndCov). Tuy nhiên, để đảm bảo khả năng so sánh giữa các lần thực thi lại khác nhau, nhóm tác giả đã loại bỏ tất cả các công cụ có hành vi không nhất quán hoặc chấm dứt trạng thái sớm do áp lực bộ nhớ trong ít nhất một lần chạy.

Kết quả cho 37 công cụ còn lại được thể hiện trong Hình 4.1. Cụ thể, **Thời gian thực thi trung vị** giảm đáng kể từ 120 phút xuống còn **13.5 phút** khi thực thi lại với cùng chiến lược khám phá và giảm hơn nữa xuống còn **8.2 phút** khi bật cơ chế cắt tỉa đường dẫn. Khi sử dụng chiến lược khám phá ít tốn kém hơn là **DFS** trong quá trình thực thi lại, thời gian thực thi trung vị tiếp tục giảm xuống còn **6.4 phút** và **5.4 phút** tương ứng.



Hình 4.1: Phân phối thời gian chạy của 37 ứng dụng coreutils

Kết quả nhóm thực nghiệm:

Experiment	Time(hr)
record	48.48633333333333
replay_dfs	10.18843333333333
replay_rc	22.48016944444444
prune_dfs	7.93247222222222
prune_rc	11.99609722222222

Hình 4.2: Tổng thời gian chạy của 37 ứng dụng coreutils

Với thời gian trung vị record là 2h, replay_dfs là 7 phút, replay_rc là 15 phút, prune_dfs là 5.8 phút, prune_rc là 8.5 phút.

4.4 Symbolic execution chạy trong thời gian dài

Trong thí nghiệm cuối cùng, nhóm tác giả tập trung vào các ứng dụng được thảo luận trong phần giới thiệu KLEE, được cấu hình với giới hạn bộ nhớ 2 GB và hoàn toàn cạn kiệt các đường dẫn trước khi đạt đến giới hạn 2 giờ. Điều này xảy ra do quá nhiều trạng thái bị chấm dứt sớm vì thiếu bộ nhớ. Trong lần chạy ghi với chiến lược RndCov ở trên, các ứng dụng này bao gồm:base64, basenc, cut, dirname, fmt, fold, head, mktemp, paste, realpath, stty, sum, tac, và wc.

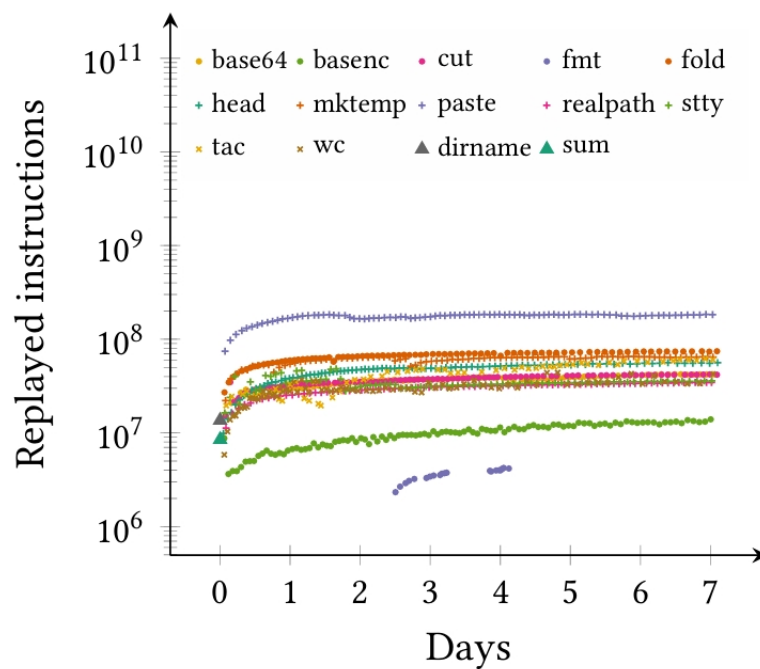
Theo trước đó, bất kể có bao nhiêu thời gian, KLEE vẫn không thể khám phá thêm các đường dẫn vượt quá một giới hạn nhất định trong những trường hợp như vậy. Mặc dù việc tăng giới hạn bộ nhớ có thể giúp ích, đây thường là một cách khắc phục không hiệu quả, vì các đường dẫn quan trọng vẫn có thể bị mất do áp lực bộ nhớ, và symbolic execution có thể hết đường dẫn vào một thời điểm sau đó.

Bắt đầu với các lần chạy có ghi nhớ từ thí nghiệm đầu tiên (symbolic execution chạy đường dẫn), nhóm tác giả thực thi lại (với cắt tỉa đường dẫn được bật) trong số 14 ứng dụng nhiều lần, sử dụng các dữ liệu đã ghi trước đó, cho đến khi tổng

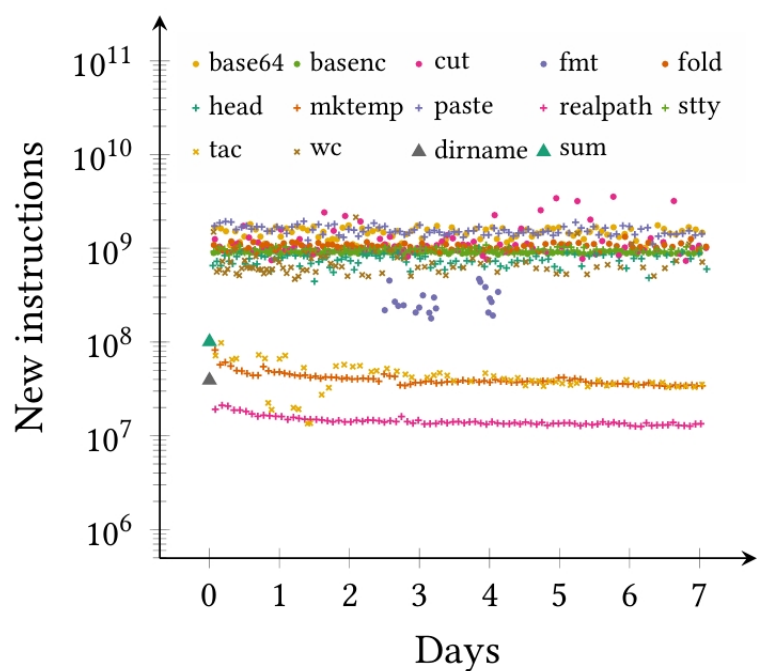
thời gian thực thi vượt quá bảy ngày. Mỗi lần khởi động lại ứng dụng, MoKlee không đặt giới hạn thời gian cho đến khi ứng dụng cạn kiệt đường dẫn một lần nữa và chấm dứt. Trong mỗi lần chạy, nhóm tác giả sử dụng heuristic mặc định **RndCov**.

Trong thiết lập này, tác giả phải khởi động lại các ứng dụng từ **20 lần** (đối với fmt) đến **105 lần** (đối với basenc), ngoại trừ hai trường hợp: dirname và sum hoàn tất bình thường (không gặp tình trạng chấm dứt đường dẫn sớm) ngay sau lần thực thi lại đầu tiên.

Hình 4.3 dưới đây hiển thị số lượng lệnh được chạy lại và số lượng lệnh mới (chưa được ghi nhớ) theo thời gian. Mỗi điểm trong biểu đồ tương ứng với một lần chạy. Các biểu đồ cho thấy phần mở rộng ghi nhớ của tác giả hoạt động hiệu quả – mỗi lần chạy lại đều có thể tiến triển trong quá trình khám phá và thực thi nhiều lệnh mới (chưa được ghi nhớ) gấp đến hai bậc so với số lượng lệnh đã được ghi nhớ.



Hình 4.3: Số lượng lệnh được chạy lại trong mỗi lần tái thực thi



Hình 4.4: Số lượng lệnh mới (chưa được ghi nhớ) trong mỗi lần tái thực thi

Kết quả nhóm thực nghiệm:

tool	NewInstructions	ReplayedInstructions
base64	16801127	3255467
basenc	92225369	3911083
cut	11254984	2609847
dirname	39133398	1360290
fmt	24302738	24302738
fold	10842850	2707044
head	72936211	5057383
mktemp	40851826	6299272
paste	15939961	1804759
realpath	13441872	3323505
stty	10257775	3439589
sum	10062603	8481294
tac	40628853	4719838
wc	52957338	3291853

Hình 4.5: Số lượng lệnh được chạy lại trong mỗi lần tái thực thi

Nhóm chạy trong 3h nhận thấy các đường đi mới tiếp tục được tìm thấy.

Chương 5

Minh họa

Do các tác giả chưa công bố mã nguồn của sản phẩm đã được phát triển mà chỉ cung cấp một gói chương trình đã được dịch, kĩ thuật được trình bày trong bài báo đã được cài đặt lại trong một công cụ kiểm thử khác để minh họa cho kĩ thuật.¹

Công cụ được sử dụng là một công cụ sinh dữ liệu kiểm thử đơn vị tự động cho ngôn ngữ TypeScript và được tự phát triển. Công cụ sử dụng phương pháp thực thi concolic[10] để thu thập dữ liệu về các đường thi hành đối với các dữ liệu đầu vào khác nhau và hệ ràng buộc cho các đường đi đó. Dữ liệu về các đường thi hành được tổng hợp trong một cấu trúc cây thực thi. Một chiến thuật tìm kiếm trên cây thực thi theo ngữ cảnh[11] của các nhánh được sử dụng để lựa chọn các đường thi hành mới cho những lần thực thi tiếp theo. Do kĩ thuật được trình bày trong bài báo cũng sử dụng cấu trúc cây thực thi, các nút trên cây thực thi đã có được bổ sung các trường dữ liệu mới cho kĩ thuật thay cho việc tạo ra một cây thực thi riêng.

Cụ thể, một trường được thêm vào mỗi nút của cây thực thi chứa thông tin của nút tương ứng trên cây ghi nhớ của kĩ thuật, bao gồm ID trong cơ sở dữ liệu, mã băm của các đoạn mã nguồn đã được thực thi, dữ liệu đầu vào cho đơn vị chương trình (sinh ngẫu nhiên hoặc là kết quả của việc giải hệ ràng buộc) và ID của các nút con. Trường dữ liệu này có thể được giải phóng khỏi nút của cây thực thi khi không còn cần thiết (bằng cách gán giá trị null). Một trường khác chứa

¹Mã nguồn của công cụ có cài đặt kĩ thuật có trên GitHub tại địa chỉ:
<https://github.com/leduyquang753/concolic-ts/tree/memoization>

cờ cho biết dữ liệu của nút ghi nhớ là mới và dữ liệu mới cũng sẽ phải được cập nhật cho các nút ghi nhớ con, điều này xảy ra khi phân kì được phát hiện. Dữ liệu của các nút ghi nhớ được lưu vào một cơ sở dữ liệu SQLite, dưới dạng một ánh xạ từ ID đến dữ liệu của các nút ghi nhớ.

Khi thực thi cho một bộ giá trị đầu vào, đối với các nút tương ứng với đường thi hành trên cây thực thi, bộ giá trị đầu vào được lưu vào mỗi nút ghi nhớ tương ứng (ngoại trừ nút gốc do nó luôn được đi qua với mọi bộ giá trị đầu vào), nếu nút ghi nhớ chưa có bộ dữ liệu đầu vào nào được lưu. Điều này là thực hiện được do công cụ hiện chưa hỗ trợ các lời gọi hàm ra môi trường bên ngoài và vì thế đường thi hành là tất định cho mỗi bộ dữ liệu đầu vào.

```
1 export function test(  
2     n: number, min: number, max: number  
3 ): number {  
4     if (n < min)  
5         return min;  
6     if (n > max)  
7         return max;  
8     return n;  
9 }
```

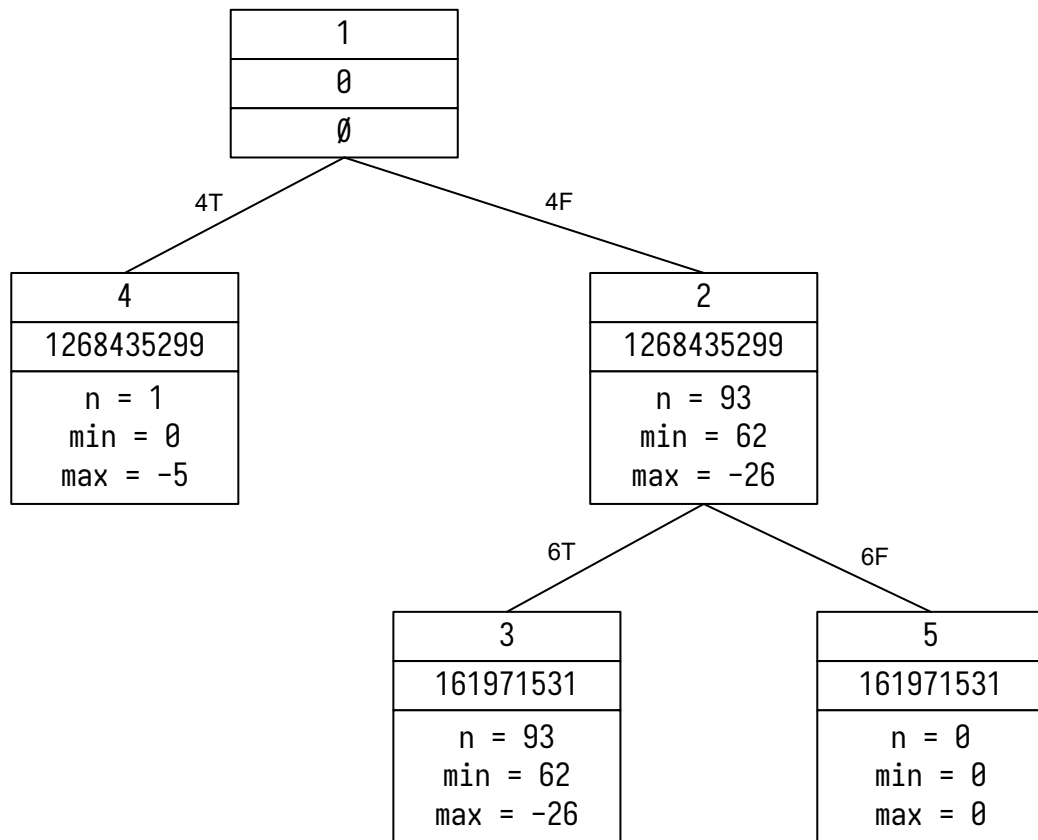
Mã nguồn 5.1: Ví dụ một hàm TypeScript minh họa cho kĩ thuật

Ví dụ một hàm minh họa được trình bày ở mã nguồn 5.1. Một lần thực hiện sinh dữ liệu kiểm thử được thực hiện với cơ sở dữ liệu ghi nhớ trống thực thi hàm, sao cho độ phủ nhánh (C_2) đạt 100%. Các đường thi hành được thực thi được trình bày trong bảng 5.1. Các con số của các nhánh trên đường thi hành tương ứng với số thứ tự của dòng trong mã nguồn.

Bảng 5.1: Các đường thi hành được thực thi trong lần chạy ban đầu

STT	Bộ giá trị đầu vào	Đường thi hành
1	n = 93 min = 62 max = -20	4F-6T
2	n = 1 min = 0 max = -5	4T
3	n = 0 min = 0 max = 0	4F-6F

Cây thực thi được ghi nhớ sau khi thực hiện sinh dữ liệu được biểu diễn trên hình 5.1. Trên mỗi nút, các ô từ trên xuống dưới lần lượt là ID (đánh số từ 1 theo thứ tự tạo nút), mã băm của mã nguồn, và bộ giá trị đầu vào được ghi nhớ cho nút.



Hình 5.1: Cây thực thi ghi nhớ sau lần chạy ban đầu

Tiếp theo, mã nguồn của hàm ví dụ được thực hiện một thay đổi nhỏ ở dòng số 6, thay toán tử < bằng toán tử <=. Hàm sau khi thay đổi được trình bày ở mã nguồn 5.2. Quá trình sinh dữ liệu kiểm thử cho hàm được thực hiện lại trong khi sử dụng dữ liệu đã được ghi nhớ từ lần thực hiện trước đó. Các đường thi hành được trình bày trong bảng 5.2 và cây thực thi được ghi nhớ sau khi thực hiện được biểu diễn trên hình 5.2, với các nút ghi nhớ có sự thay đổi dữ liệu được tô xám.

```

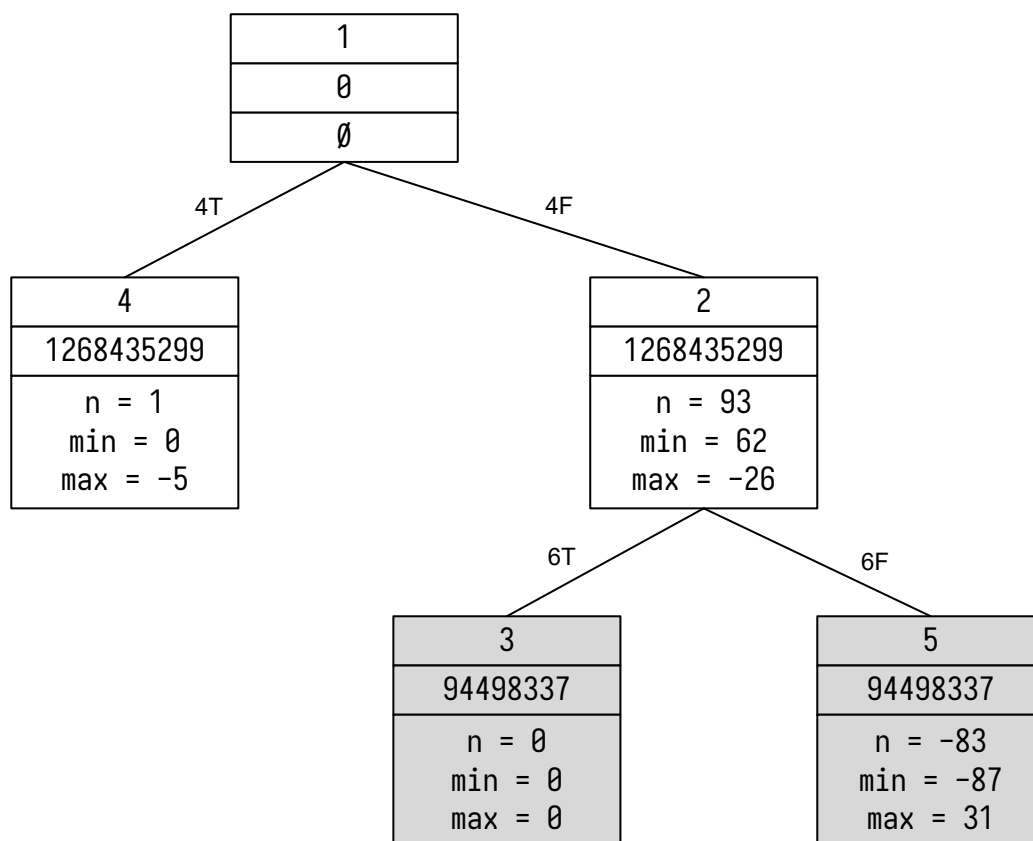
1 export function test(
2     n: number, min: number, max: number
3 ): number {
4     if (n < min)
5         return min;
6     if (n >= max)
7         return max;
8     return n;
9 }

```

Mã nguồn 5.2: Hàm TypeScript sau khi đã thay đổi

Bảng 5.2: Các đường thi hành được thực thi trong lần chạy thứ hai

STT	Bộ giá trị đầu vào	Đường thi hành	Tái sử dụng
1	n = -83 min = -87 max = 31	4F-6F	Không
2	n = 1 min = 0 max = -5	4T	Có
3	n = 0 min = 0 max = 0	4F-6T	Không



Hình 5.2: Cây thực thi ghi nhớ sau lần chạy thứ hai

Ở câu lệnh điều kiện thứ nhất ở dòng số 4, công cụ đã tái sử dụng bộ giá trị đầu vào để thực hiện đường thực thi qua nhánh chưa được phủ do mã nguồn không có sự thay đổi tính đến vị trí của câu lệnh đó. Tuy nhiên, ở câu lệnh điều kiện thứ hai ở dòng số 6, công cụ đã phát hiện có sự thay đổi mã nguồn và thực hiện giải lại hệ ràng buộc để sinh bộ giá trị đầu vào tiếp theo, đồng thời cập nhật dữ liệu trong các nút ghi nhớ tương ứng với ID là 3 và 5.

Chương 6

Kết luận

Dựa trên việc ghi nhớ, được tăng cường bằng phát hiện phân nhánh, thay đổi search heuristic và lưu trữ hiệu quả giúp bảo tồn cấu trúc của cây thực thi, có thể chạy vô thời hạn thực thi tượng trưng trên các ứng dụng lớn trong khi tiếp tục khám phá các con đường mới. Tuy nhiên, bài báo vẫn còn nhiều vấn đề:

1. Chưa nói cụ thể về kết quả bộ giải (solver results) được lưu gồm chính xác những nội dung gì.
2. Không giải thích rõ về nhận xét cho phương pháp cũ rằng phải dùng cùng heuristics cho các lần chạy.
3. Mã nguồn được hứa hẹn đăng nhưng chưa được thực hiện sau 4 năm. (Chỉ có một số lần tên MoKlee được nhắc đến trên <https://github.com/klee/klee>.)
4. Chưa giải thích rõ ảnh hưởng của phương pháp lên cung cấp góc nhìn tổng quát của chương trình.
5. Tác giả có nhắc đến lựa chọn đường đi ưu tiên để tải lên bộ nhớ tuy nhiên chưa giải thích các chương trình nhận biết đường đi ưu tiên.
6. Không xét đến trường hợp xảy ra vòng lặp vô hạn khi tạo vector hướng nhánh.

Phụ lục

Các câu hỏi phản biện và trả lời phản biện

Câu hỏi 1: Ý nghĩa của từ “instructions” trong kết quả thực nghiệm là gì?

Trả lời: Do công cụ của các tác giả thực hiện kiểm thử trên mã máy đã được biên dịch của các chương trình mà không phải là mã nguồn, “instructions” là các chỉ dẫn mã máy được thực thi trong chương trình.

Câu hỏi 2: Nếu chương trình có các vòng lặp vô hạn, chúng được xử lý như thế nào?

Trả lời: Công cụ của các tác giả đặt ra một giới hạn về độ sâu của cây thực thi, khi một vòng lặp được lặp quá nhiều lần thì sẽ đạt đến độ sâu giới hạn và cây thực thi sẽ được cắt tại vị trí này, quá trình thực thi tượng trưng chuyển sang các nhánh theo các hướng khác.

Câu hỏi 3: Tại sao chiến thuật tìm kiếm theo chiều sâu lại tiết kiệm bộ nhớ hơn chiến thuật tìm kiếm theo chiều rộng?

Trả lời: Với độ sâu n , độ phức tạp bộ nhớ của chiến thuật tìm kiếm theo chiều rộng là $O(2^n)$, trong khi độ phức tạp bộ nhớ của chiến thuật tìm kiếm theo chiều sâu chỉ là $O(n)$, tức thấp hơn rất nhiều.

Câu hỏi 4: Kỹ thuật được trình bày có giải quyết vấn đề hết bộ nhớ khi thực hiện thực thi tương trưng hay không?

Trả lời: Kỹ thuật chỉ giảm thiểu đáng kể lượng dữ liệu được lưu trong bộ nhớ tại một thời điểm trong quá trình kiểm thử chứ không cho phép quá trình có thể kéo dài vô hạn mà không bao giờ hết bộ nhớ. Trong thực nghiệm của các tác giả, công cụ được cài đặt để tự động khởi động lại quá trình thực thi tương trưng khi hết bộ nhớ, và những lần thực thi tiếp theo có thể khám phá những phần khác

của cây thực thi. Tuy nhiên, nhờ việc này mà quá trình thực thi tượng trưng có thể khám phá cây thực thi rộng và sâu hơn so với các kĩ thuật khác.

Câu hỏi 5: Yếu tố khó khăn nhất của kĩ thuật được trình bày là gì?

Trả lời: Yếu tố khó khăn nhất của kĩ thuật là việc phát hiện phân kì. Có nhiều yếu tố có thể thay đổi trong môi trường thực thi của các chương trình và bài báo cũng nêu lên hạn chế rằng công cụ của các tác giả chưa thể phát hiện hết các yếu tố phân kì. Cụ thể, công cụ chưa phát hiện được các phân kì mà không làm thay đổi dòng điều khiển, và các phân kì trong đó mã bẫy bị trùng nhau.

Câu hỏi 6: Nếu chỉ thực hiện thực thi tượng trưng, tại sao lại có yếu tố môi trường thực thi?

Trả lời: Các tác giả đề cập không chỉ thực thi tượng trưng đơn thuần mà còn là thực thi tượng trưng kết hợp với thực thi cụ thể, hay còn gọi là thực thi concolic[10]. Chương trình được thực thi cụ thể được cho phép tương tác với môi trường thực thi bên ngoài một cách bình thường.

Tài liệu tham khảo

- [1] Elvira Albert, Puri Arenas, Miguel Gómez-Zamalloa, Germán Puebla, and Manuel V. Hermenegildo, *Test case generation by symbolic execution: Basic concepts, a clp-based instance, and actor-based concurrency*, Formal Methods for Executable Software Models: 14th International School on Formal Methods for the Design of Computer, Communication, and Software Systems, Advanced Lectures (Marco Bernardo et al., eds.), Lecture Notes in Computer Science, vol. 8483, Springer, 2014, pp. 263–309.
- [2] Thanassis Avgerinos, Sang Kil Cha, Brent Lim Tze Hao, and David Brumley, *Automatic exploit generation*, Communications of the ACM **57** (2014), no. 2, 74–84.
- [3] Frank Busse, Martin Nowack, and Cristian Cadar, *Running symbolic execution forever*, Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis, 2020, pp. 63–74.
- [4] Cristian Cadar, Daniel Dunbar, and Dawson R. Engler, *Klee: Unassisted and automatic generation of high-coverage tests for complex systems programs*, Proceedings of the 8th USENIX Conference on Operating Systems Design and Implementation, OSDI'08, vol. 8, USENIX Association, 2008, pp. 209–224.
- [5] Cristian Cadar, Daniel Dunbar, Dawson R Engler, et al., *Klee: unassisted and automatic generation of high-coverage tests for complex systems programs.*, OSDI, vol. 8, 2008, pp. 209–224.
- [6] Sang Kil Cha, Thanassis Avgerinos, Alexandre Rebert, and David Brumley, *Unleashing mayhem on binary code*, 2012 IEEE Symposium on Security and Privacy, IEEE Computer Society, May 2012, pp. 380–394.

- [7] Pär Emanuelsson and Ulf Nilsson, *A comparative study of industrial static analysis tools*, Electronic notes in theoretical computer science **217** (2008), 5–21.
- [8] Gordon Hogenson, Genevieve Warren, Tim Sherer, et al., *Overview of microsoft intellitest*, 2017–2022, Accessed: 2023-01-23.
- [9] James C King, *Symbolic execution and program testing*, Communications of the ACM **19** (1976), no. 7, 385–394.
- [10] Koushik Sen, Darko Marinov, and Gul Agha, *Cute: a concolic unit testing engine for c*, Proceedings of the 10th European Software Engineering Conference held jointly with the 13th ACM SIGSOFT International Symposium on Foundations of Software Engineering (ESEC/FSE-13), 2005, pp. 263–264.
- [11] Hyunmin Seo and Sunghun Kim, *How we get there: a context-guided search strategy in concolic testing*, Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE 2014), 2014, pp. 413–424.
- [12] Guowei Yang, Corina S Păsăreanu, and Sarfraz Khurshid, *Memoized symbolic execution*, Proceedings of the 2012 International Symposium on Software Testing and Analysis, 2012, pp. 144–154.